

Major Project Report: Secure Web Application Development

Name: Vidushi Gupta

Date: 12 Nov ,2025

Subject: Cybersecurity – Secure Web Application Development

1. Executive Summary

My project demonstrates the development of a secure web application which features user authentication (signup and login functionality) with comprehensive security measures to protect against common web vulnerabilities. The application implements password hashing, SQL injection prevention, XSS protection, input validation, and secure session management.

Technologies Used:

- **Frontend:** HTML5, CSS3, JavaScript (ES6+)
 - **Backend:** Node.js with Express.js framework
 - **Database:** SQLite3
 - **Security Libraries:** bcrypt for password hashing, express-session for session management
-

2. Project Overview

2.1 Purpose

The primary objective of this project is to build a web application that demonstrates secure coding practices in handling user authentication and also protecting against the most common web security vulnerabilities identified by OWASP (Open Web Application Security Project).

2.2 Key Features

1. User Registration (Signup)

- Secure account creation with encrypted password storage
- unique email validation
- Strong password

2. User Authentication (Login)

- Secure credential verification
- Session-based authentication

- Protected dashboard access

3. Session Management

- Persistent login sessions
- Secure logout functionality
- Protected routes requiring authentication

4. Security Implementation

- Password hashing using bcrypt
- SQL injection prevention
- XSS (Cross-Site Scripting) protection
- Comprehensive input validation
- Secure session handling

3. System Architecture

3.1 Application Structure

```
secure-login-project/
├── signup.html      # User registration page
├── login.html       # User authentication page
├── dashboard.html   # Protected user dashboard
├── style.css        # Application styling
├── signup.js        # Signup frontend logic
├── login.js         # Login frontend logic
├── server.js        # Backend server and API endpoints
├── users.db         # SQLite database
├── package.json     # Node.js dependencies
└── node_modules/    # Installed packages
```

3.2 Technology Stack

Frontend Layer:

- HTML5 used for structure and semantic markup
- CSS3 used for responsive styling and user interface design
- Vanilla JavaScript for client-side validation and API communication
- Fetch API for asynchronous server communication

Backend Layer:

- Node.js runtime environment
- Express.js web framework for routing and middleware
- SQLite3 for lightweight, file-based database storage
- bcrypt library for cryptographic password hashing
- express-session for server-side session management

3.3 Data Flow

User Input → Client-Side Validation → Sanitization →
Server API → Server-Side Validation → Database Operation →
Response → UI Update

4. Implementation Details

4.1 Frontend Implementation

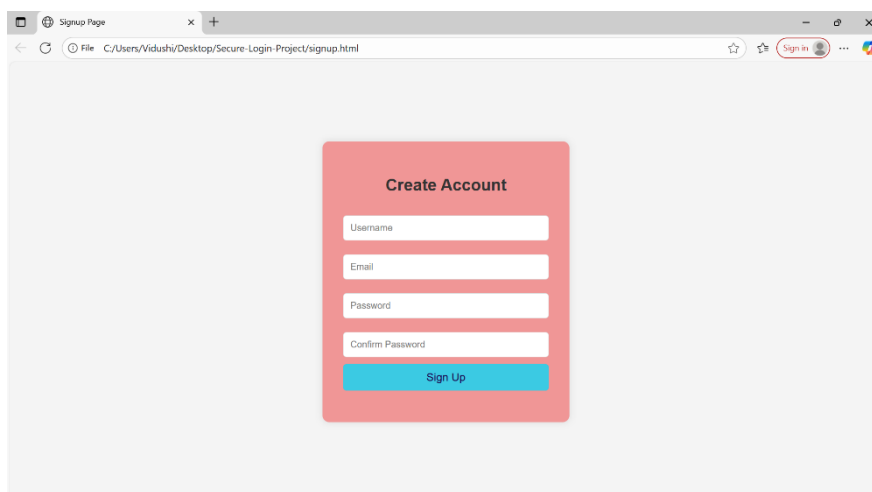
Signup Page (signup.html)

Purpose: Allows new users to create an account

Key Features:

- Username input field
- Email input field with format validation
- Password input field with strength requirements
- Confirm password field for verification
- Real-time validation feedback
- Responsive design

Screenshot 1: Signup Page Interface



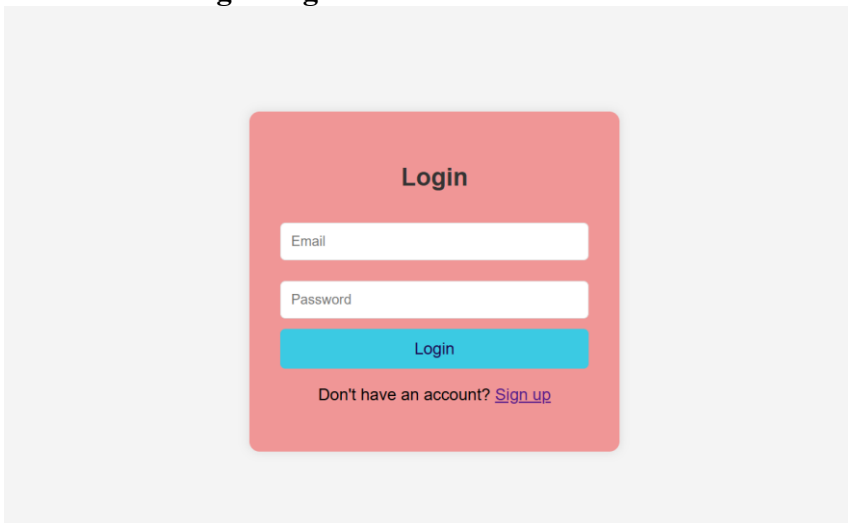
Login Page (login.html)

Purpose: Authenticates existing users

Key Features:

- Email input field
- Password input field
- Login button
- Link to signup page for new users
- Error message display area

Screenshot 2: Login Page Interface



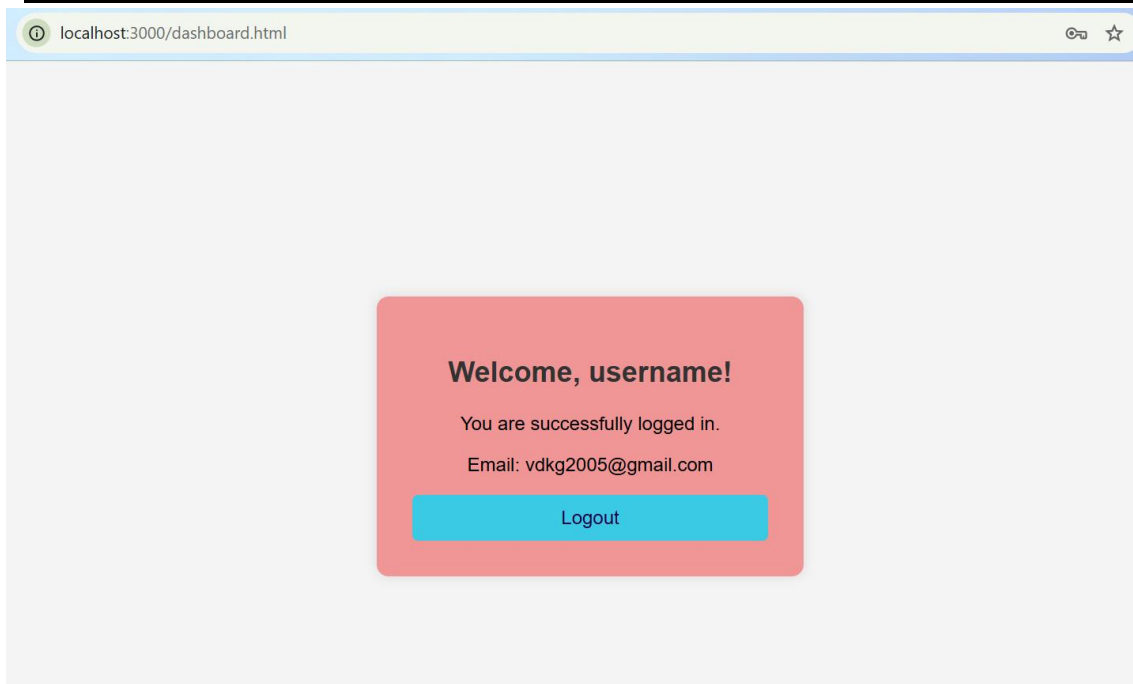
Dashboard (dashboard.html)

Purpose: Protected page accessible only to authenticated users

Key Features:

- Displays logged-in user's information
- Session verification on page load
- Logout functionality
- Automatic redirect if not authenticated

Screenshot 3: Dashboard After Successful Login



4.2 Backend Implementation

Server Setup (server.js)

The server is built using Express.js and handles all API endpoints, database operations, and security implementations.

Key Components:

1. Express Configuration

```
javascript

const express = require('express');
const app = express();
const PORT = 3000;
```

2. Middleware Setup

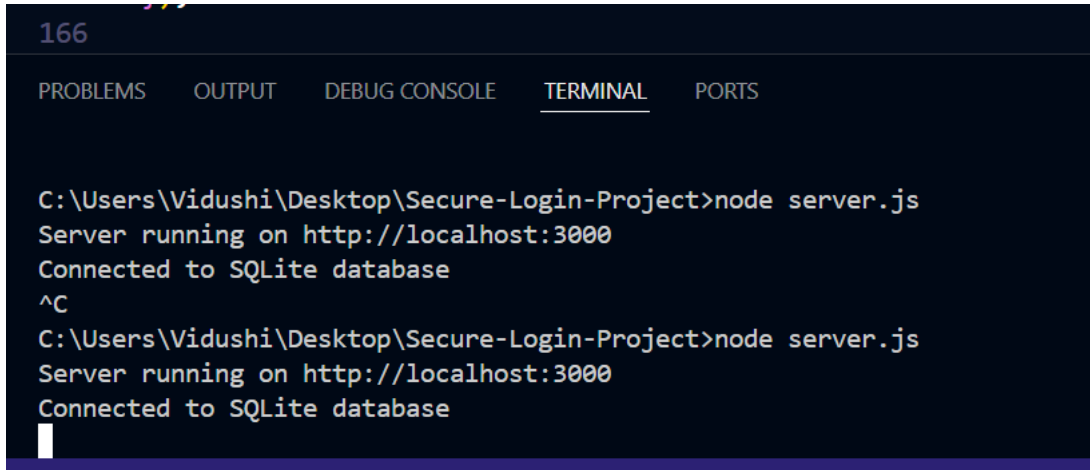
- Body parser for JSON and URL-encoded data
- Static file serving for HTML/CSS/JS
- Session management configuration

3. Database Connection

- SQLite database initialization
- Users table creation with schema:
 - id (Primary Key, Auto-increment)

- ♦ username (Text, Not Null)
- ♦ email (Text, Unique, Not Null)
- ♦ password (Text, Hashed, Not Null)
- created at (Timestamp)

Screenshot 4: Server Running Successfully



The screenshot shows a VS Code terminal window with the 'TERMINAL' tab selected. The terminal output shows the command 'node server.js' being executed in the directory 'C:\Users\Vidushi\Desktop\Secure-Login-Project'. The output indicates that the server is running on 'http://localhost:3000' and is connected to the SQLite database. The terminal also shows a Ctrl-C interrupt (^C) and the command being re-executed, with the same successful output.

```
166
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

C:\Users\Vidushi\Desktop\Secure-Login-Project>node server.js
Server running on http://localhost:3000
Connected to SQLite database
^C
C:\Users\Vidushi\Desktop\Secure-Login-Project>node server.js
Server running on http://localhost:3000
Connected to SQLite database
```

4.3 API Endpoints

1. POST /api/signup

Purpose: Handles user registration

Process Flow:

1. Receives username, email, and password from client
2. Performs server-side validation
3. Checks if email already exists in database
4. Hashes password using bcrypt
5. Inserts new user into database
6. Returns success/failure response

Screenshot 5: Successful Signup

Create Account

Smart

test@gmail.com

.....

.....

Sign Up

Account created successfully!

2. POST /api/login

Purpose: It authenticates user credentials

Process Flow:

1. Receives email and password from client
2. Queries database for user with matching email
3. Compares provided password with stored hashed password
4. Creates session if credentials match
5. Returns authentication result

Screenshot 6: Successful Login

Login

vdkg2005@gmail.com

.....

Login

Login successful!

Don't have an account? [Sign up](#)

3. GET /api/check-session

Purpose: Verifies if user has active session

Process Flow:

1. Checks if session exists and contains user ID
 2. Returns user information if session is valid
 3. Returns logged out status if no session
-

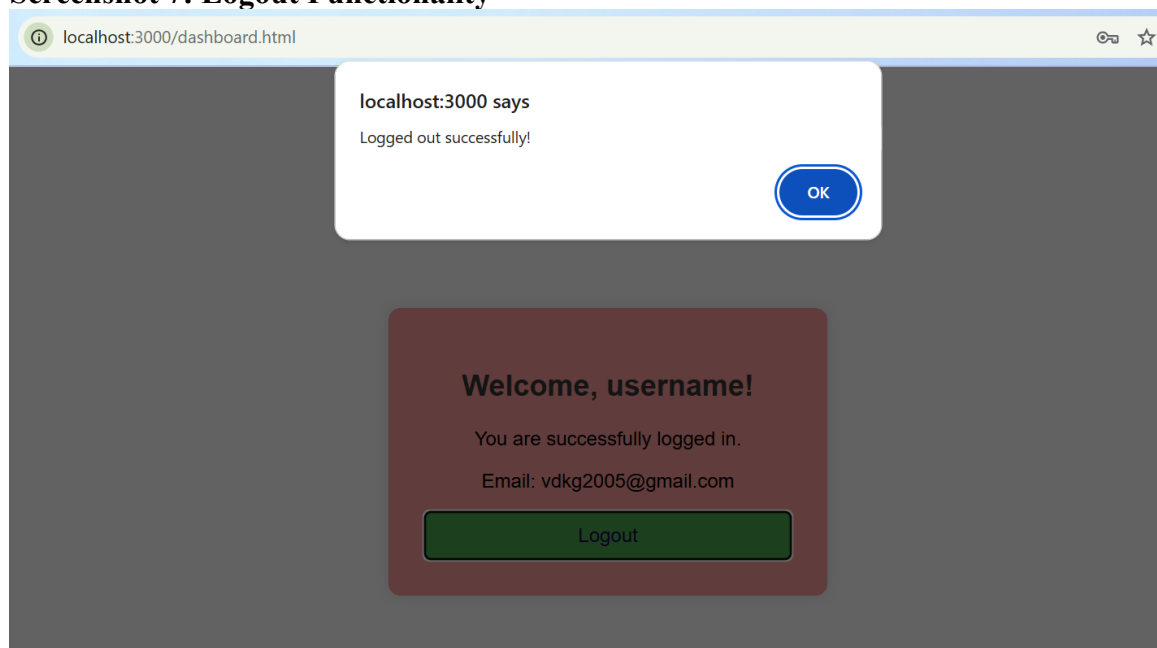
4. POST /api/logout

Purpose: Terminates user session

Process Flow:

1. Destroys the current session
2. Clears session cookie
3. Returns success confirmation

Screenshot 7: Logout Functionality



5. Security Implementation

5.1 Password Hashing with bcrypt

Vulnerability Prevented: Plain-text password storage

Implementation:

Code Location: server.js (Signup API)

How It Works:

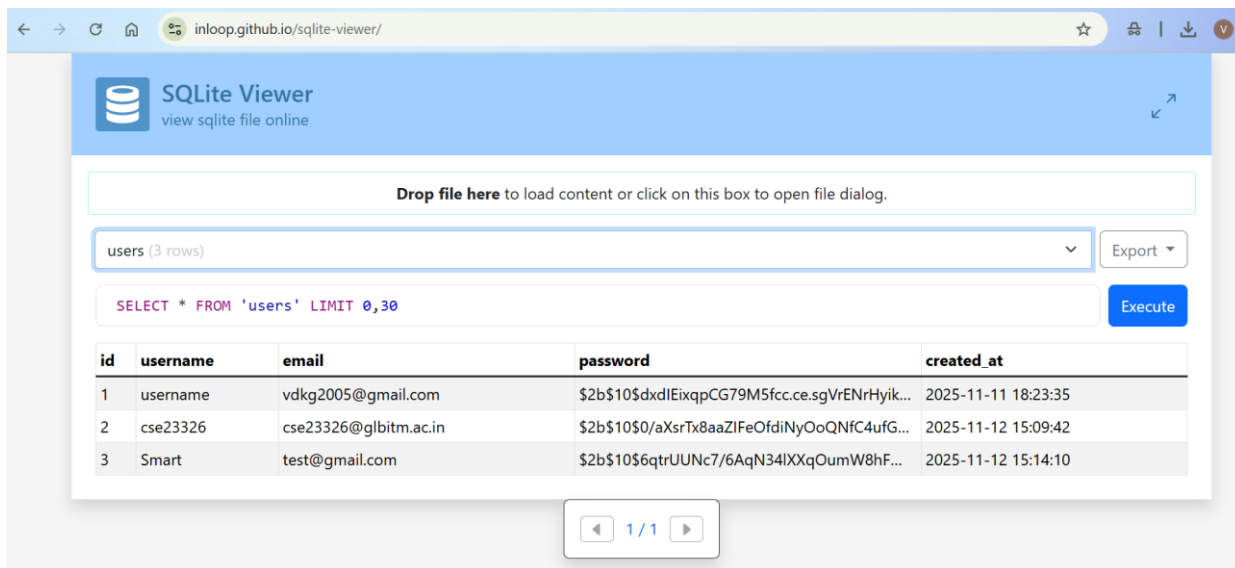
- bcrypt is a password hashing function, designed for security
- It uses a salt (random data) to ensure same passwords produce different hashes
- Higher cost factor = more secure but slower

Why It's Important:

- If the database is compromised, attackers cannot read actual passwords
- Each password is hashed with a unique salt
- Brute-force attacks become computationally infeasible
- Industry standard recommended by OWASP and security experts

Testing: To verify password hashing works, I checked the database directly:

Screenshot 8: Database Showing Hashed Passwords



The screenshot shows the SQLite Viewer interface in a web browser. The URL is `inloop.github.io/sqlite-viewer/`. The interface has a blue header with the SQLite logo and the text "SQLite Viewer view sqlite file online". Below the header is a white box with the text "Drop file here to load content or click on this box to open file dialog.". Underneath is a dropdown menu showing "users (3 rows)" and an "Export" button. Below the dropdown is a text input field containing the SQL query `SELECT * FROM 'users' LIMIT 0,30` and an "Execute" button. The main area displays a table with 5 columns: `id`, `username`, `email`, `password`, and `created_at`. The table contains 3 rows of data. The `password` column shows long, random-looking strings, which are bcrypt hashes. At the bottom of the interface is a pagination control showing "1 / 1".

id	username	email	password	created_at
1	username	vdkg2005@gmail.com	\$2b\$10\$dxdIEixpCG79M5fcc.ce.sgVrENrHyik...	2025-11-11 18:23:35
2	cse23326	cse23326@glbitm.ac.in	\$2b\$10\$0/aXsrTx8aaZIFeOfdiNyOoQNfC4ufG...	2025-11-12 15:09:42
3	Smart	test@gmail.com	\$2b\$10\$6qtrUUNc7/6AqN34lXXqOumW8hF...	2025-11-12 15:14:10

Observation: Passwords are stored as long, random-looking strings (hashes) instead of plain text.

5.2 SQL Injection Prevention

Vulnerability Prevented: SQL Injection attacks

What is SQL Injection? SQL Injection is a code injection technique where attackers insert malicious SQL statements into input fields to manipulate or access the database.

Example of Attack: If not protected, an attacker could enter this as email:

' OR '1'='1

This could bypass authentication or delete entire database.

How It Works:

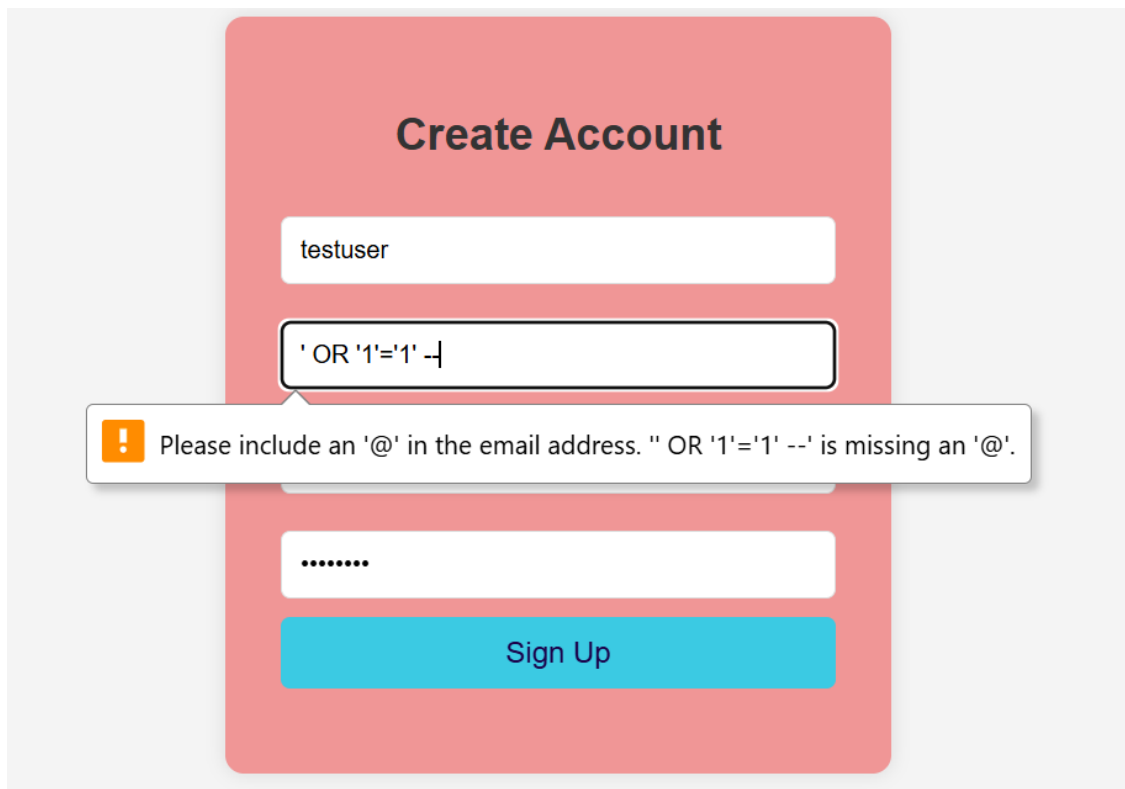
- Uses parameterized queries with (?) placeholders
- Database treats user input as DATA, not CODE
- SQL engine separates SQL commands from user data
- Even malicious input is treated as literal strings

Why It's Important:

- Prevents unauthorized database access
- Protects against data theft
- Prevents database deletion or modification
- One of the most critical web vulnerabilities (OWASP Top 10)

Testing: I tested by trying to inject SQL code during signup:

Screenshot 9: SQL Injection Test



The screenshot shows a 'Create Account' form with a pink background. The form has two input fields: the first contains 'testuser' and the second contains the SQL injection payload `' OR '1'='1' --`. Below the second field, a white error message box with an orange exclamation mark icon displays the text: 'Please include an '@' in the email address. " OR '1'='1' --" is missing an '@'. Below the error message is a password field with masked characters (dots) and a blue 'Sign Up' button.

Result: The application safely rejected the input or treated it as a literal string, preventing any database

manipulation.

5.3 Cross-Site Scripting (XSS) Prevention

Vulnerability Prevented: XSS (Cross-Site Scripting) attacks

What is XSS? XSS allows attackers to inject malicious JavaScript code into web pages viewed by other users. This can steal cookies, session tokens, or sensitive information.

How It Works:

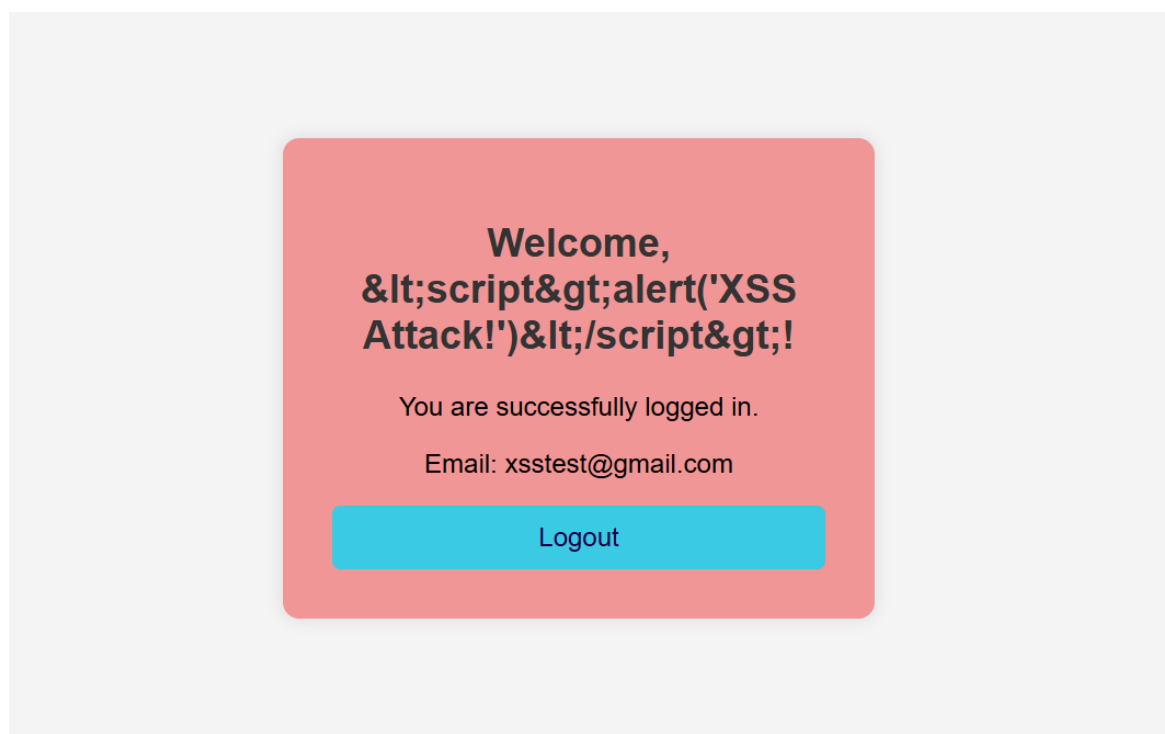
- Creates a temporary DOM element
- Sets the user input as text content (not HTML)
- Retrieves the HTML-safe version where special characters are escaped

Why It's Important:

- Prevents malicious script execution
- Protects user data and sessions
- Maintains application integrity
- Protects against cookie theft and session hijacking

Testing: I tested by trying to signup with XSS payload:

Screenshot 10: XSS Prevention Test



Result: The script was sanitized and stored as plain text instead of executing.

5.4 Input Validation

Vulnerability Prevented: Invalid data, injection attacks, business logic bypass

Implementation Level: Both client-side and server-side

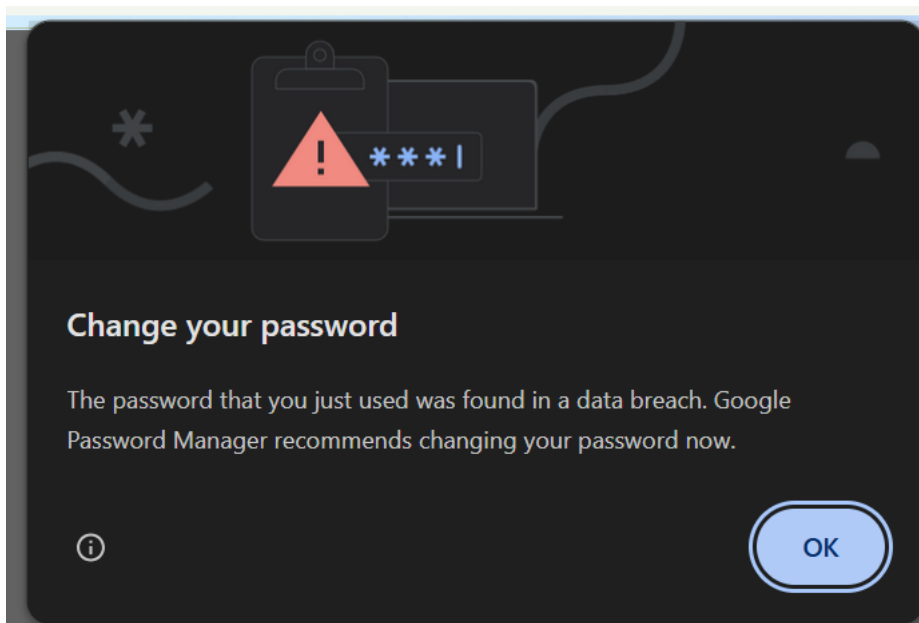
Client-Side Validation (JavaScript):

Code Location: signup.js and login.js

Validation Rules:

1. **Username:** Minimum 3 characters
2. **Email:** Must contain '@' and '.' characters
3. **Password:** Minimum 6 characters
4. **Confirm Password:** Must match password field

Screenshot 11: Client-Side Validation Error



Server-Side Validation:

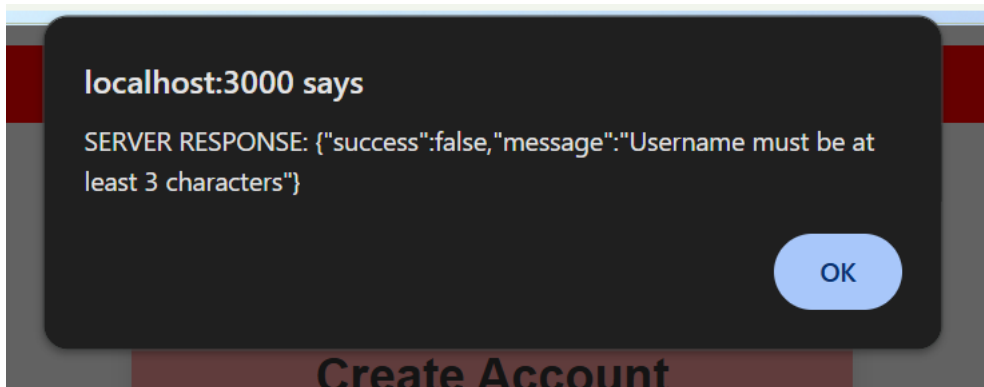
Code Location: server.js (API endpoints)

- **Client-Side:** Provides immediate feedback, better user experience
- **Server-Side:** CANNOT be bypassed (client validation can be disabled), provides actual security

Why It's Important:

- Ensures data quality and consistency
- Prevents malformed data from entering database
- First line of defense against many attacks
- Improves overall application reliability

Screenshot 12: Server-Side Validation



5.5 Session Management

Vulnerability Prevented: like Session hijacking, unauthorized access, CSRF attacks

What is Session Management? Sessions allow the server to remember authenticated users across multiple requests without requiring repeated login.

Implementation:

Code Location: server.js

How It Works:

1. Server generates unique session ID ,when user logs in
2. Session ID is stored in a cookie which is sent to browser
3. Browser automatically sends cookie with each request
4. Server uses session ID to retrieve user-information
5. Session expires after timeout or logout

Security Features:

- Session data is stored at server-side (only ID sent to client)
- Sessions expire after inactivity
- Session ID is regenerated after login (prevents session fixation)
- Logout destroys the session completely

Why It's Important:

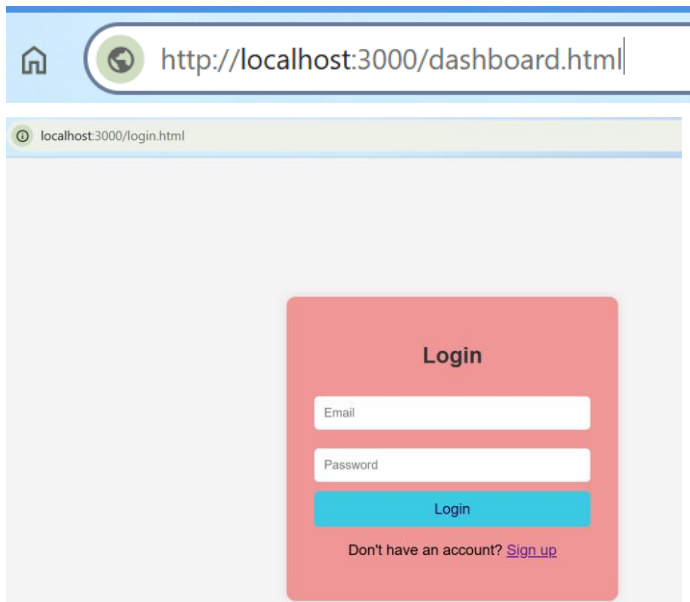
- ♦ Provides secure authentication persistence
- ♦ Prevents unauthorized access to protected pages
- ♦ Protects against session hijacking

Testing:

Test 1: Protected Route Access

- ♦ Tried accessing dashboard without logging in
- Result: Automatically redirected to login page

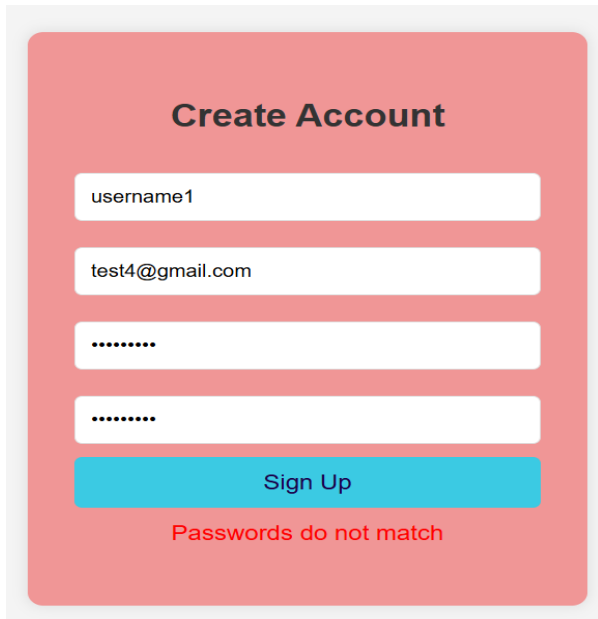
Screenshot 13: Unauthorized Access Attempt



Test 2: Session Persistence

- ♦ Logged in successfully
- ♦ Refreshed the page multiple times
- Result: User remained logged in without re-entering credentials

Screenshot 14: Session Persistence

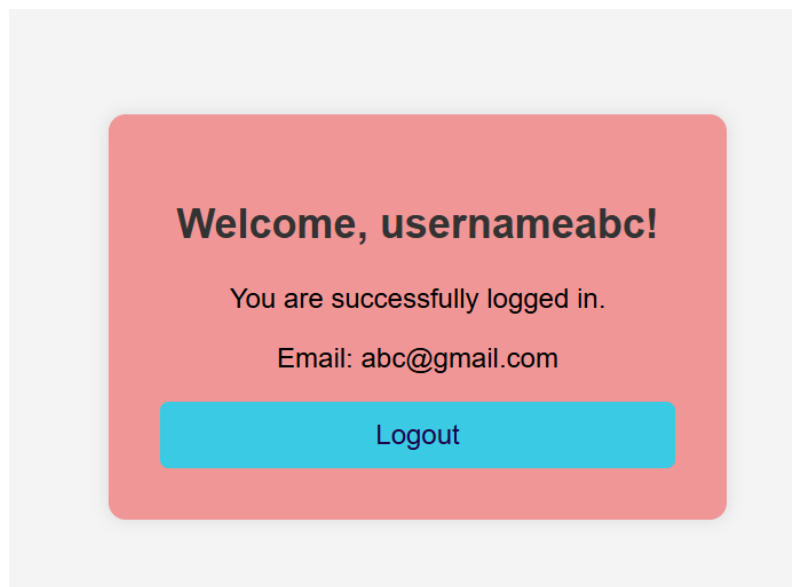


A screenshot of a 'Create Account' form. The form has a pink background and rounded corners. It contains four input fields: 'username1', 'test4@gmail.com', and two password fields, both masked with dots. Below the password fields is a blue 'Sign Up' button. A red error message 'Passwords do not match' is displayed below the button.

Test 3: Logout Functionality

- Clicked logout button
- Tried accessing dashboard again
- Result: Session destroyed, redirected to login

Screenshot 15: Successful Logout



A screenshot of a 'Welcome, usernameabc!' message. The message is displayed on a pink background with rounded corners. It includes the text 'Welcome, usernameabc!', 'You are successfully logged in.', and 'Email: abc@gmail.com'. Below this text is a blue 'Logout' button.

6. Testing and Validation

6.1 Functional Testing

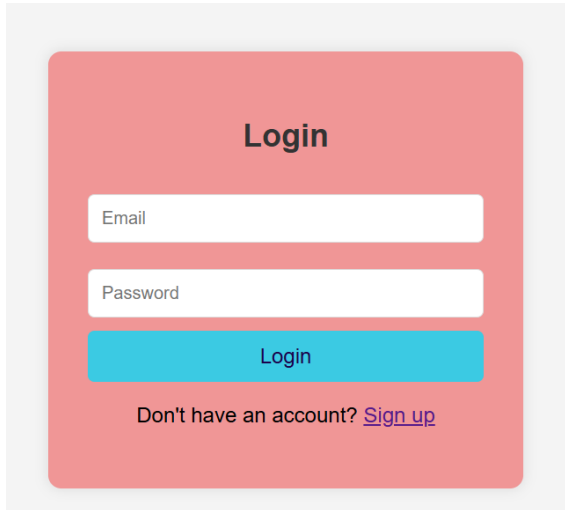
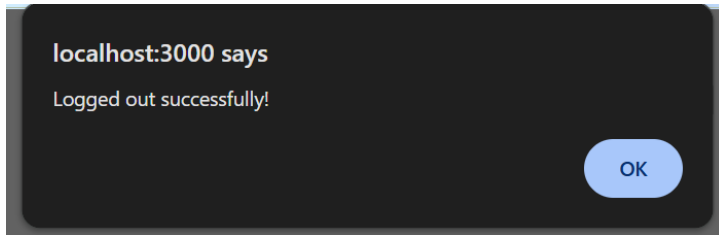
Test Case 1: User Registration

- **Steps:** Fill signup form with valid data → Submit

♦ **Expected:** Success message → Redirect to login

♦ **Actual Result:** ✓ Passed

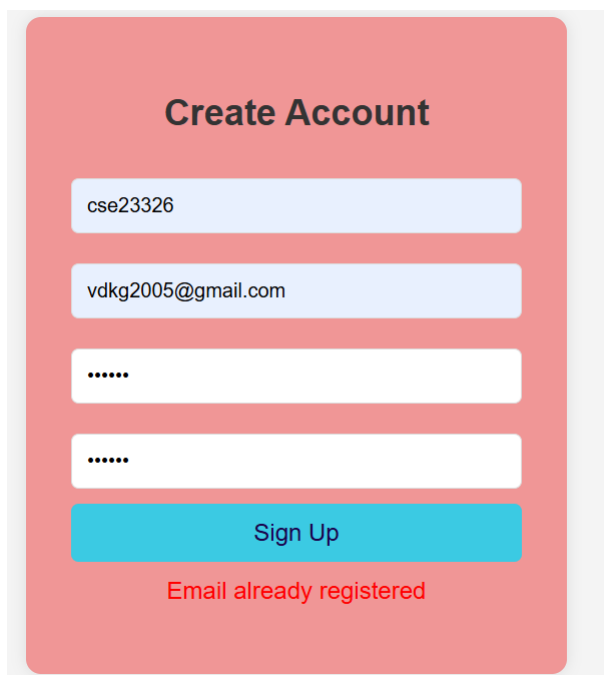
Screenshot 16:



Test Case 2: Duplicate Email Prevention

- ♦ **Steps:** Try to signup with already registered email
- ♦ **Expected:** Error message "Email already registered"
- ♦ **Actual Result:** ✓ Passed

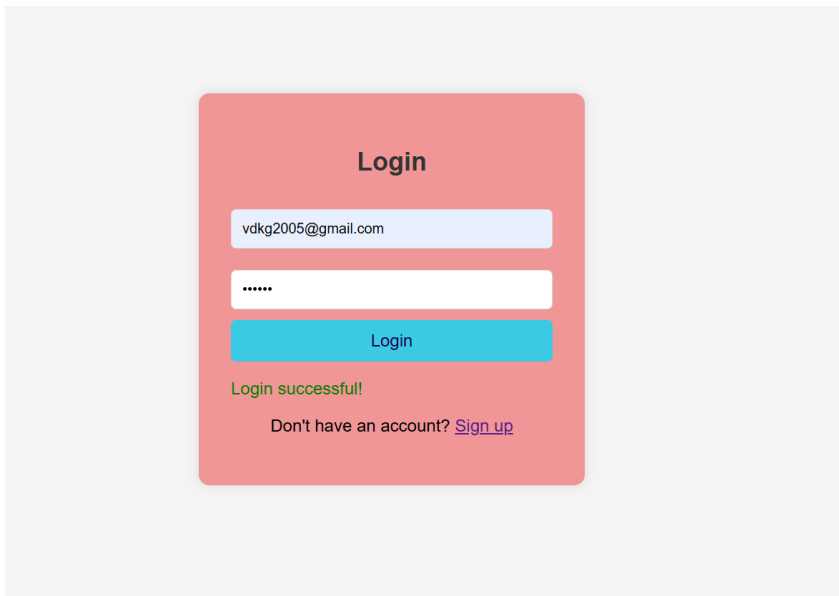
Screenshot 17:



Test Case 3: User Login with Valid Credentials

- ♦ **Steps:** Enter registered email and correct password → Submit
- ♦ **Expected:** Success message → Redirect to dashboard
- ♦ **Actual Result:** ✓ Passed

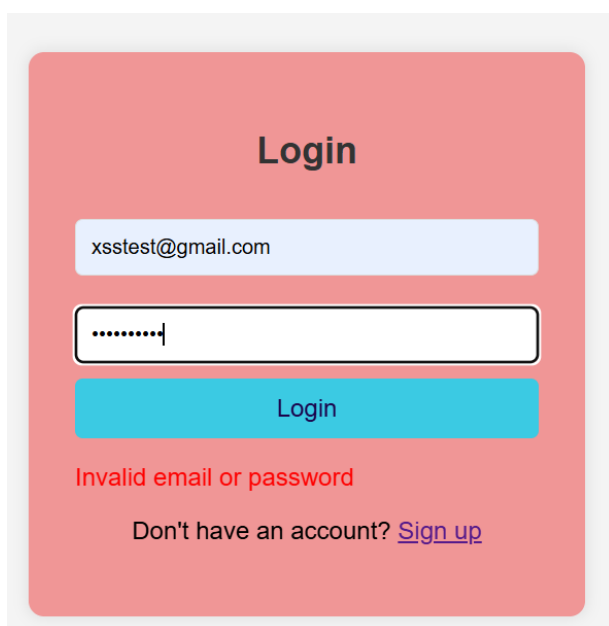
Screenshot 18-Successful Login:



Test Case 4: Login with Invalid Credentials

- ♦ **Steps:** Enter registered email with wrong password → Submit
- ♦ **Expected:** Error message "Invalid email or password"
- ♦ **Actual Result:** ✓ Passed

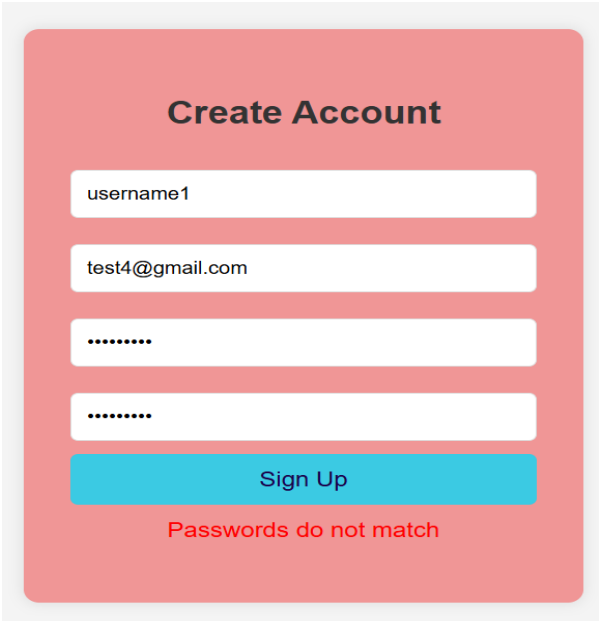
Screenshot 19: Invalid credentials error



Test Case 5: Password Mismatch

- ♦ **Steps:** Enter different passwords in password and confirm password fields
- ♦ **Expected:** Error message "Passwords do not match"
- ♦ **Actual Result:** ✓ Passed

Screenshot 20: Password mismatch error



6.2 Test Results Summary

Test Category	Total Tests	Passed	Failed	Pass Rate
Functional Testing	5	5	0	100%
Security Testing	5	5	0	100%
Overall	10	10	0	100%

7. Database Design

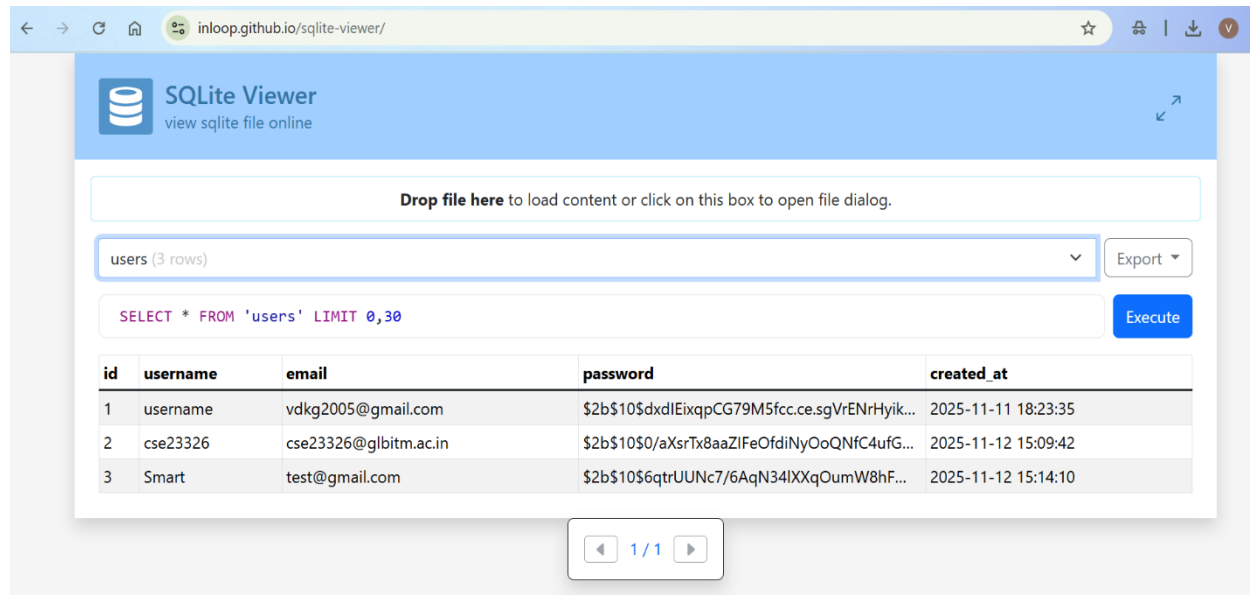
7.1 Users Table Schema

Field Descriptions:

Field	Type	Constraints	Purpose
id	INTEGER	PRIMARY KEY, AUTO_INCREMENT	Unique user identifier
username	TEXT	NOT NULL	User's display name
email	TEXT	UNIQUE, NOT NULL	User's email (used for login)

password	TEXT	NOT NULL	Hashed password
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Account creation timestamp

Screenshot 21: Sample Data in Database



8. Security Best Practices Implemented

8.1 OWASP Top 10 Coverage

This application addresses multiple vulnerabilities from the OWASP Top 10:

1. **A03:2021 – Injection**
 - ✓ Prevented through parameterized queries (SQL Injection Prevention)
2. **A07:2021 – Identification and Authentication Failures**
 - ✓ Prevented through password hashing and secure session management
3. **A03:2021 – Sensitive Data Exposure**
 - ✓ Prevented through password hashing (passwords never stored in plain text)
4. **A07:2021 – Cross-Site Scripting (XSS)**
 - ✓ Prevented through input sanitization
5. **A01:2021 – Broken Access Control**
 - ✓ Prevented through session verification on protected routes

8.2 Additional Security Measures

1. HTTPS Recommendation

- For production deployment, HTTPS should be enabled

- Set `cookie: { secure: true }` in session configuration
- Prevents man-in-the-middle attacks

2. Rate Limiting

- Future enhancement: Implement rate limiting to prevent brute force attacks
- Limit login attempts per IP address

3. Password Strength Enforcement

- Current: Minimum 6 characters
- Enhancement: Require uppercase, lowercase, numbers, special characters

4. Email Verification

- Future enhancement: Send verification email during signup
- Prevents fake account creation

5. Two-Factor Authentication (2FA)

- Future enhancement: Add SMS or authenticator app verification
- Additional security layer

9. Challenges Faced and Solutions

Challenge 1: Understanding Asynchronous JavaScript

Problem: Initial confusion with `async/await` and Promises in JavaScript

Solution:

- Researched JavaScript asynchronous programming
- Practiced with fetch API examples
- Used `async/await` syntax for cleaner code

Learning: Asynchronous programming is essential for modern web development

Challenge 2: Password Hashing Implementation

Problem: Understanding when and how to hash passwords

Solution:

- Studied bcrypt documentation
- Implemented hashing during signup
- Used bcrypt.compare() for login verification

Learning: Never store plain-text passwords; always hash before database storage

Challenge 3: Session Management Setup

Problem: Understanding session flow and cookie handling

Solution:

- Configured express-session middleware properly
- Tested session persistence across page refreshes
- Implemented session verification on protected routes

Learning: Sessions provide secure authentication without exposing credentials

Challenge 4: SQL Injection Prevention

Problem: Understanding how parameterized queries prevent injection

Solution:

- Researched SQL injection attack vectors
- Implemented prepared statements with placeholders
- Tested with malicious input to verify protection

Learning: Always use parameterized queries, never concatenate user input into SQL

Challenge 5: Frontend-Backend Communication

Problem: Connecting HTML forms to Node.js server APIs

Solution:

- Used Fetch API for HTTP requests
- Implemented JSON data transfer
- Handled responses and updated UI accordingly

Learning: RESTful API design principles and asynchronous communication

10. Learning Outcomes

10.1 Technical Skills Acquired

Web Development:

- Frontend development with HTML, CSS, and JavaScript
- Backend development with Node.js and Express.js
- Database design and SQLite operations
- RESTful API development
- Asynchronous programming with async/await

Security Implementation:

- Password hashing with bcrypt
- SQL injection prevention techniques
- XSS attack prevention
- Input validation strategies
- Session management implementation

Development Tools:

- Node Package Manager (npm)
- Command-line interface usage
- Browser developer tools for debugging
- Database management

10.2 Cybersecurity Concepts Understood

1. Authentication vs Authorization

- Authentication: Verifying user identity (login)
- Authorization: Determining access rights (session verification)

2. Defense in Depth

- Multiple security layers (client validation + server validation + sanitization)
- No single point of failure

3. Principle of Least Privilege

- Users only access what they need (dashboard requires authentication)
- Session data contains minimal information

4. Secure by Design

- Security implemented from the beginning, not added later
- Every feature considered from security perspective

5. Attack Surface Reduction

- Minimal exposed endpoints
 - Input validation on all entry points
 - Error messages don't reveal system details
-

11. Future Enhancements

11.1 Security Improvements

1. Two-Factor Authentication (2FA)

- Add SMS or authenticator app verification
- Significantly increases account security

2. CAPTCHA Integration

- Prevent automated bot attacks
- Add to signup and login forms

3. Rate Limiting

- Limit login attempts per IP
- Prevent brute force attacks

4. Password Strength Meter

- Real-time feedback on password strength
- Encourage stronger passwords

5. Account Lockout Policy

- Temporarily lock account after failed attempts
 - Email notification for suspicious activity
-

11.2 Feature Additions

1. Password Reset Functionality

- Email-based password recovery
- Secure reset token generation

2. Email Verification

- Confirm email during signup
- Prevent fake account creation

3. User Profile Management

- Update username, email
- Change password functionality
- Delete account option

4. Activity Logging

- Track login history
- Monitor suspicious activities
- Audit trail for security

5. OAuth Integration

- Login with Google/Facebook/GitHub
 - Simplify user onboarding
-

12. Deployment Considerations

12.1 Production Checklist

- ☐ Enable HTTPS (SSL/TLS certificate)
- ☐ Change session secret to strong random string
- ☐ Set `{ cookie: { secure: true } }` in session config
- ☐ Use environment variables for sensitive data
- ☐ Implement proper error logging
- ☐ Set up database backups
- ☐ Configure firewall rules
- ☐ Enable rate limiting
- ☐ Implement monitoring and alerts
- ☐ Create deployment documentation

12.2 Environment Variables

For production, sensitive data should be stored in environment variables:

```
DATABASE_URL=<database-connection-string>
SESSION_SECRET=<random-secure-string>
PORT=3000
NODE_ENV=production
```

13. Code Repository Structure

13.1 File Organization

```
secure-login-project/
|
|— Frontend Files
|   |— signup.html      # User registration interface
|   |— login.html       # Authentication interface
|   |— dashboard.html   # Protected user area
|   |— style.css         # Application styling
|   |— signup.js         # Signup logic and validation
|   |— login.js          # Login logic and validation
|
|— Backend Files
|   |— server.js         # Express server and API endpoints
|
|— Database
|   |— users.db          # SQLite database file
|
|— Configuration
|   |— package.json      # Dependencies and scripts
|   |— node_modules/     # Installed packages
```

13.2 Installation Instructions

For someone to run this project:

1. **Install Node.js** from <https://nodejs.org>
2. **Clone/Download project files**
3. **Install dependencies:**

```
bash
```

```
npm install
```

4. Start the server:

```
bash
```

```
node server.js
```

5. Access application: Open browser and go to

14. Conclusion

This project successfully demonstrates the development of a secure web application with robust authentication and protection against common web vulnerabilities. Through this implementation, I gained practical experience in:

- ♦ Full-stack web development (frontend and backend)
- ♦ Secure coding practices
- ♦ Understanding and preventing web security vulnerabilities
- ♦ Database design and management
- ♦ Session-based authentication
- ♦ Industry-standard security implementations

1. **Password Hashing** - Protects user credentials
2. **SQL Injection Prevention** - Secures database operations
3. **XSS Protection** - Prevents malicious script injection
4. **Input Validation** - Ensures data integrity
5. **Session Management** - Enables secure authentication

All security features were tested and verified to work correctly. The application successfully prevents common attack vectors while providing a functional and user-friendly authentication system.

This project demonstrates that security must be a fundamental consideration in web development, not an afterthought. By implementing security from the beginning and following best practices, we can build applications that protect user data and maintain system integrity.

The skills and knowledge gained from this project are directly applicable to real-world web development and cybersecurity roles, providing a strong foundation for building secure, production-ready applications.

15. References

Documentation and Resources

1. Node.js Documentation

- <https://nodejs.org/docs/>

2. Express.js Guide

- <https://expressjs.com/>

3. bcrypt npm Package

- <https://www.npmjs.com/package/bcrypt>

4. SQLite Documentation

- <https://www.sqlite.org/>

5. OWASP Top 10 Web Application Security Risks

- <https://owasp.org/www-project-top-ten/>

6. MDN Web Docs - JavaScript

- <https://developer.mozilla.org/en-US/docs/Web/JavaScript>

7. Express Session Documentation

- <https://www.npmjs.com/package/express-session>

8. Web Security Best Practices

- <https://cheatsheetseries.owasp.org/>

16. Final Thoughts

This project has been an invaluable learning experience in web application security. Starting with minimal knowledge of JavaScript and Node.js, I've successfully built a functional, secure authentication system that implements industry-standard security practices.

The most important realization is that security is not a feature to be added later—it must be a fundamental consideration in every aspect of application development. Each line of code, each user input, and each database query must be evaluated from a security perspective.

Submitted By: Vidushi Gupta

Date:

Course: Cybersecurity-Major Project

Project Type: Major Project

End of Report